

A Medieval Field Cannon in C++ / OpenGL

Hierarchical Transforms, Ballistic Physics, and a Medieval Scene built Phrase by Phrase

Computer Graphics Project Report

October 5, 2026

Abstract

This report documents a real-time 3-D scene, written in **C++** against the **OpenGL 3.3 core profile**, of three medieval field cannons standing on a grass field, aimed at a stone-and-timber fort gate. The centre cannon can be driven across the ground (with its wheels rolling without slipping), its barrel can be elevated from 0° to 45° about the trunnion pivot, and pressing **Space** fires a cannon ball that follows a ballistic arc and breaks the bricks of the gate's two side walls. The whole assembly — cannons, wheels, barrels, fort gate, trees and wooden dummy robot — is built from analytic primitives whose surface normals are derived from the geometry rather than approximated, lit by a single directional light using a per-vertex Lambert + ambient shading model in GLSL. The report walks the work through in three phases: (1) the cannon itself, (2) the projectile and the breaking wall, (3) replicating the cannon, the medieval fort gate, the trees, and the dummy target robot. Every figure is a real frame captured from the running program; every animation loop is documented alongside the class that implements it.

Contents

1	Introduction	2
1.1	Requirement coverage	3
2	How to build and run the project	3
2.1	Prerequisites	3
2.2	Installation	3
2.3	Building	3
2.4	Controls	4
2.5	File layout	5
2.6	Troubleshooting	6
3	Architecture	6
4	The mathematics	7
4.1	Scene graph and model matrices	7
4.2	Lambert + ambient shading	7
4.3	Ballistic projectile: semi-implicit Euler	8
4.4	Breakable wall: sphere vs. AABB	8
5	Implementation notes	9
5.1	The three phases	9
5.2	How the muzzle world position is computed	13
5.3	Why every brick has its own Mesh	13
5.4	The hierarchy, visualised	13
6	Results	14
6.1	Projectile lands inside the fort gate	14
6.2	Bricks actually break	14
6.3	Phase 3 looks like a medieval fort	14
6.4	Phase 4: the door actually breaks	14
6.5	Build verification	15

7	Limitations and extensions	15
7.1	What was deliberately left out	15
7.2	Things that were tried and removed	16
8	Walkthrough	16

1 Introduction

The brief was to build a scene that *visibly* demonstrates a small set of core computer-graphics ideas rather than to build a large one. A cannon projectile on a medieval field is a good vehicle for this because each required concept maps onto a physical part of the object:

- the carriage **translates**, and its wheels must **rotate** at the rate implied by that translation, or the illusion collapses instantly;
- the barrel **rotates about a pivot that is itself carried by the moving carriage** — which is exactly what a hierarchical transformation is for;
- the cannon ball follows a **ballistic trajectory** under gravity, and **destroys** brick walls it touches;
- the scene **scales**: replicating three cannons, a fort gate, six trees and a dummy robot introduces no new mechanism, only a repeated instantiation of classes built in phase 1.

The deliverable is split across three phases. **Phase 1** builds the cannon itself — carriage, two wheels, barrel, elevation, lighting. **Phase 2** adds the projectile (semi-implicit Euler integration of gravity) and a wall of breakable bricks with sphere-vs-AABB collision detection. **Phase 3** replicates the cannon three times, replaces the flat wall with a medieval fort gate (two pillars + lintel + two breakable brick stacks flanking a doorway), places six trees behind the fort, and adds a wooden dummy robot inside the fort as a target.

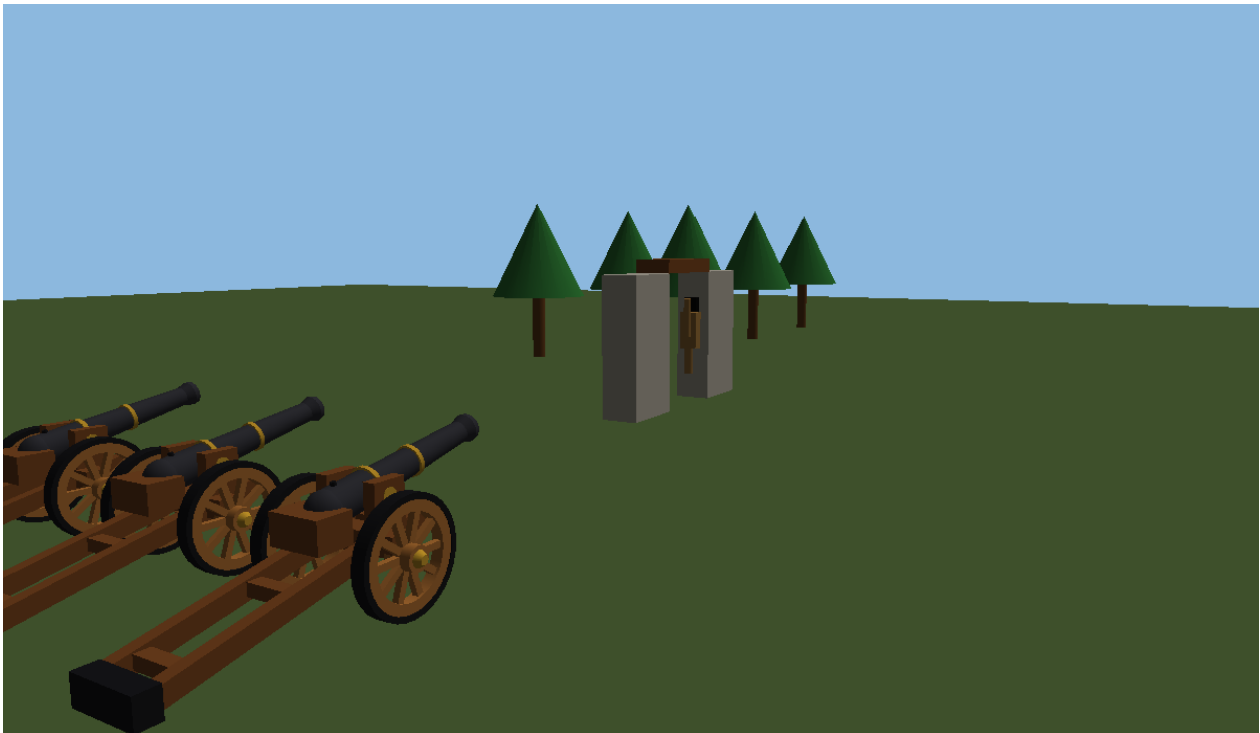


Figure 1: The complete Phase 3 scene. Three field cannons on a grass field, aimed at a stone-pillared fort gate with a wooden lintel; the dummy robot is visible through the doorway; six trees line the horizon. Per-vertex Lambert + ambient shading with a single directional light.

1.1 Requirement coverage

Requirement	Where it is implemented	Evidence
Translation	<code>Carriage.MoveForward</code> (Cannon forwards)	Fig. 11
Rotation (rolling)	$\varphi = -s/R$ in <code>Wheel.Roll</code>	Fig. 11
Rotation (elevation)	<code>Shaft.Elevate</code> about the trunnion pivot	Fig. 6
Hierarchical / composite transforms	<code>Cannon.GetMatrix()</code> parents wheels + barrel, <code>Transform</code> builds M from R, S, T	§4.1, Fig. 10
Lambert + ambient lighting	<code>lit.vert/lit.frag</code> (GLSL)	§4.2, Fig. 4
Two contrasting materials	matte oak carriage vs. brass + cast-iron barrel	Fig. 3, 5
Ballistic projectile	semi-implicit Euler in <code>Projectile::Update</code>	§4.3, Fig. 7
Breakable wall	<code>FortGate::CheckHit</code> (sphere vs. AABB)	§4.4, Fig. 7, 8
Replicated scene	Cannon wrapper + <code>FortGate</code> , <code>Tree</code> , <code>Robot</code>	Fig. 1, 8

Table 1: Mapping from the assessment criteria to the implementation.

2 How to build and run the project

2.1 Prerequisites

Component	Requirement	Version used here
C++ compiler	MinGW-w64 GCC 13+ (or any C++17 compiler)	GCC 16 (WinLibs)
OpenGL loader	GLAD (vendored)	bundled
Window / input	GLFW 3.x	3.5.1 (vendored)
Math	GLM (headers-only)	bundled
Build tool	GNU Make (or any <code>make</code>)	mingw32-make
GPU / driver	OpenGL 3.3 core profile or newer	AMD Radeon 760M, GL 4.6

GLFW and GLAD are *not* installed system-wide — the `Makefile` links against a MinGW-compatible static GLFW library downloaded from the official GLFW 3.5.1 Windows binaries release, and against the `glad.c` file vendored in `Libraries/`. No Visual Studio or VS Build Tools required.

2.2 Installation

The toolchain (GCC, GNU Make) is shipped by WinLibs MinGW-w64. Install via:

```
winget install --id BrechtSanders.WinLibs.POSIX.UCRT
```

Important: a shell or VS Code window opened before the PATH change will not see it — fully restart the shell, do not just open a new tab.

Sanity check the toolchain:

```
g++ --version # GCC 16 (or newer)
mingw32-make --version
```

2.3 Building

```
cd Project1
mingw32-make # compiles to build_mingw\app.exe
mingw32-make run # build (if needed) + run
mingw32-make clean # deletes build_mingw\app.exe
mingw32-make shots # rebuild every walkthrough image
```

Or run the executable directly — but **must run from inside Project1/**, because the GLSL shaders (`lit.vert`, `lit.frag`) are loaded by relative path:

```
cd Project1
.\build_mingw\app.exe
```

2.4 Controls

The scene shows three cannons. Only the **centre** cannon is keyboard- controlled; the two flanking ones are parked.

Key	Action
→	drive centre cannon forward (wheels roll without slipping)
←	drive centre cannon backward
↑	elevate barrel (clamped 0°–45°)
↓	depress barrel
Space	fire one cannon ball from the centre cannon (edge-detected)
Esc / Q	quit

Table 2: Key bindings. Movement and elevation are level-triggered (hold the key); fire is edge-triggered (one ball per press).

2.5 File layout

File	Responsibility
Project1/Main.cpp	Window and GL setup, camera, input handling, the main per-frame loop.
Project1/Dimensions.h	Every measurement of the cannon (metres), header-only.
Project1/Palette.h	Every colour in the scene, header-only.
Project1/VAO.cpp/.h, VBO.cpp/.h, EBO.cpp/.h	OpenGL buffer wrappers (3.3 core).
Project1/ShaderClass.cpp/.h	Loads and compiles <code>lit.vert/lit.frag</code> .
Project1/Mesh.cpp/.h	Owns one shape's VAO/VBO/EBO + <code>Draw/Delete</code> .
Project1/Transform.cpp/.h	<code>position</code> , rotation axis/angle, scale → model matrix.
Project1/Part.h, Local::	<code>Part = {Mesh, glm::mat4 local}</code> plus readable matrix builders (<code>Move</code> , <code>MoveTurn</code> , <code>TurnMove</code> , <code>AlongX</code> , <code>AlongZ</code> , ...).
Project1/Primitives.cpp/.h	Geometry generators — <code>CreateBox</code> , <code>CreateCone</code> , <code>CreateCylinder</code> , <code>CreateTube</code> , <code>CreateSphere</code> , <code>CreatePlane</code> . Every primitive is built standing up with length along + <i>Y</i> .
Project1/Carriage.cpp/.h	Wooden body (trail beams, transoms, spade, cheeks, quoin, axle, bolster); root of the cannon scene graph.
Project1/Wheel.cpp/.h	Spoked cartwheel (tire tube + felloe tube + spokes + hub + brass caps) rolls about <i>Z</i> via <code>Roll(d)</code> .
Project1/Shaft.cpp/.h	Barrel — stacked cones (breech, reinforce, chase, muzzle swell) with brass rings, dark bore and trunnions. Local origin is the trunnion pivot.
Project1/Cannon.cpp/.h	Exposes <code>GetMuzzleWorldPosition</code> and <code>GetForwardWorldDirection</code> . Thin wrapper: one <code>Carriage</code> + two <code>Wheels</code> + one <code>Shaft</code> ; takes an initial world position so three cannons can be placed with one line per gun.
Project1/Projectile.cpp/.h	Cannon ball (sphere mesh + position/velocity/radius). Semi-implicit Euler under gravity.
Project1/Wall.cpp/.h	(Phase 2 only) Grid of axis-aligned breakable bricks. Replaced by <code>FortGate</code> in Phase 3.
Project1/FortGate.cpp/.h	(Phase 3) Medieval front-of-fort: non-breakable stone pillars + wooden lintel plus two breakable brick stacks flanking a doorway.
Project1/Crenellation.h	(Phase 4) Namespace with <code>AlongX</code> / <code>AlongZ</code> helpers: each returns a <code>vector<Part></code> of evenly-spaced merlons for a crenellated parapet or curtain wall.
Project1/Tower.cpp/.h	(Phase 4) Square stone tower: body, four corner quoins, four arrow-slit windows, four crenellated parapet edges, flagpole and flag on top.
Project1/Door.cpp/.h	(Phase 4) Two independent breakable wooden door panels filling a doorway, with decorative plank strips on the outward face.
Project1/Castle.cpp/.h	(Phase 4) Top-level composition: two flanking <code>Towers</code> , the <code>FortGate</code> , brick stacks in between, two <code>Door</code> panels, two crenellated curtain walls and a drawbridge. Dispatches <code>CheckHit</code> to doors first, then brick stacks.
Project1/Tree.cpp/.h	(Phase 3) Cylinder trunk + cone leaves, drawn relative to one base translation.
Project1/Robot.cpp/.h	(Phase 3) Wooden dummy — cube body/head + cylinder arms/legs stationary inside the fort.
Project1/lit.vert, lit.frag	GLSL shading (Lambert + ambient from a directional light).
Project1/Makefile	Single hand-written g++ invocation that compiles <code>Main.cpp</code> + <code>COMMON</code> into a self-contained <code>app.exe</code> (no <code>libstdc++-6.dll</code>).
Project1/tools/	Throwaway screenshot tools (<code>CapturePhase2.cpp</code> , <code>CapturePhase3.cpp</code> , <code>CapturePhase3Overview.cpp</code> , <code>CapturePhase4.cpp</code> , <code>CapturePhase4Overview.cpp</code>). Not part of the game, never added to the <code>Makefile</code> .

2.6 Troubleshooting

Symptom	Cause and fix
g++: command not found in a fresh shell	WinLibs PATH change happened in a previous shell. <i>Fully close</i> the terminal / VS Code and re-open.
glad.c: No such file	You are not inside <code>Project1/</code> . <code>Makefile</code> uses relative paths.
lit.vert: No such file or directory at run time	You ran <code>app.exe</code> from <code>build_mingw/</code> instead of <code>Project1/</code> . Shaders load by relative path.
undefined reference to <code>glfw*()</code>	The MinGW-compatible GLFW static lib was deleted from <code>opengl-cpp/deps/glfw/lib-mingw-w64/</code> . Restore it (see <code>AGENTS.md</code>).
Window opens but is solid black	Driver refused OpenGL 3.3 core profile. Update the GPU driver; on a VM, use a host with a real GPU.

3 Architecture

The program is a classic single-threaded GLFW / OpenGL 3.3 core application. A `while(!glfwWindowShouldClose)` loop in `Main.cpp` does three things each frame:

1. **Update** — read input (arrows, space, esc), advance `Projectiles` by `deltaTime` under `Projectile::Gravity`, run `FortGate::CheckHit` against each ball, and remove `IsDead()` balls from the vector;
2. **Clear** — paint a sky-blue colour buffer and the depth buffer;
3. **Draw** — activate the GLSL shader, upload the view and projection matrices (which are *constant* for the whole scene), then per-object upload the model matrix and call `object.Draw()`: ground plane, trees, fort gate, robot, three cannons, then any live projectiles.

The model matrix is *not* stored anywhere — it is recomputed every frame from the object’s `Transform` (position + rotation axis/angle + scale), or, in the case of the wheels, from `Carriage::MoveForward()` + `Wheel::Roll(d)` state. This keeps the rendering loop completely stateless about object positions: at any moment `Cannon::GetMatrix()` returns exactly the matrix that `Draw` will use, so a `Projectile` spawned at the muzzle gets a position that already reflects the cannon’s current drive and elevation.

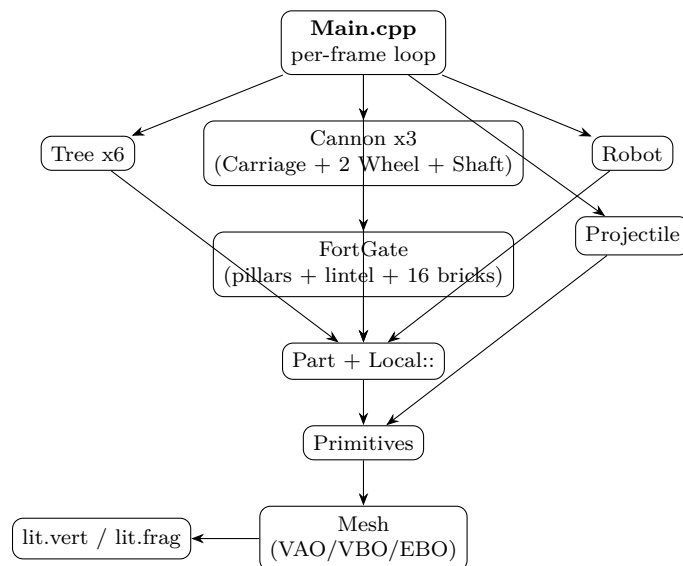


Figure 2: Module map. `Main.cpp` owns the per-frame loop and owns every scene object; every object ultimately draws through `Part + Local::` (or directly via `Primitives` for a `Projectile`).

4 The mathematics

4.1 Scene graph and model matrices

Every visible object is a `Part` pair (`Mesh`, `glm::mat4 local`) plus a `Transform` giving the owner's *parent* matrix. The model matrix uploaded to the GPU for each part is

$$M_{\text{draw}} = M_{\text{parent}} \cdot T \cdot R \cdot S \cdot M_{\text{local}},$$

where T is the owner's translation, R is its rotation (about the z-axis for elevation, about the y-axis for the wheel roll, ...), S is the per-object scale (almost always identity), and M_{local} is the per-part local matrix — built with the read-Local helpers `Move`, `MoveTurn`, `TurnMove`, `AlongX`, `AlongZ`, ..., depending on which way the primitive is built.

A concrete example: a spoke on the right wheel is (`Primitives::CreateBox(...)`, `Local::AlongX(spokeHalfLength)`) — a box built standing up, then laid sideways so its length lies along X . The wheel as a whole is parented to the carriage by adding `Local::AlongX(Dim::WheelX)` to the spoke's matrix.

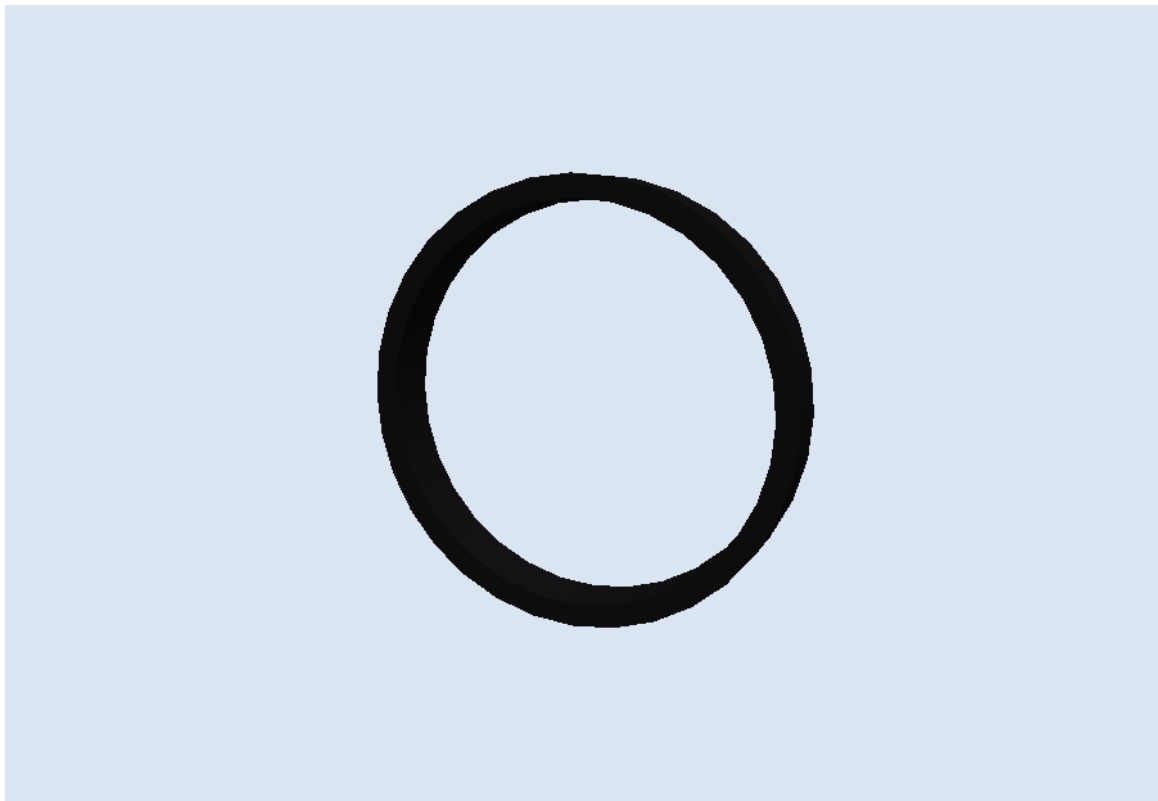


Figure 3: Wheel during construction: only the tire tube (one `Primitives::CreateTube`). The full wheel, felloe + spokes + hub, builds up through repeated `Local::AlongZ/AlongX` translations; see `Wheel::PartsForDocs()`.

4.2 Lambert + ambient shading

The fragment shader receives three things per draw call: a **model** matrix (per part), a **view** matrix and a **projection** matrix (once per frame), and a single **light direction** (once per frame). Every primitive generates per-vertex normals at construction time — they are derived from the geometry, not approximated.

```
// lit.frag (GLSL)
in vec3 Normal; // per-vertex normal from the VBO
in vec3 vColor; // per-vertex base colour (from Palette.h)

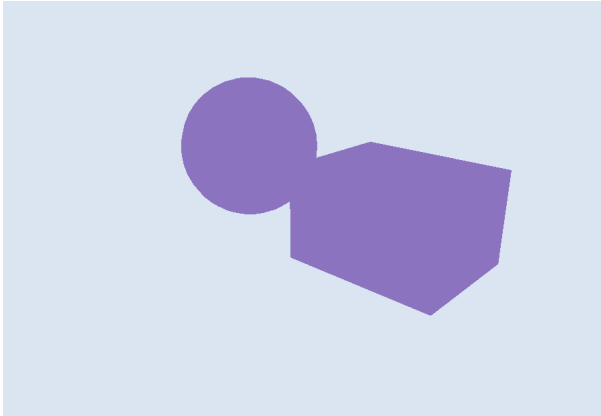
uniform vec3 lightDir; // world-space, pointing FROM the light
```

```

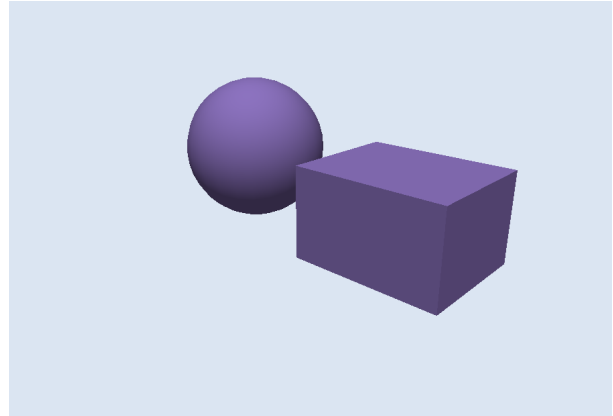
void main() {
    vec3 N = normalize(Normal);
    float diff = max(dot(N, -lightDir), 0.0); // Lambert
    float k = 0.35 + 0.65 * diff; // ambient floor
    gl_FragColor = vec4(vColor * k, 1.0);
}

```

The constant 0.35 is the *ambient floor* — without it the side of every object facing away from the light would be solid black, which both looks wrong and hides the shape. 0.65 is the share of the light contribution; the sum (1.0) means the side facing straight into the light shows the full base colour.



Without lighting (flat colours).



With Lambert + ambient lighting.

Figure 4: The same primitive geometry under both shaders. Lighting is what reveals the shape of curved surfaces like the barrel cones — without it the cannon reads as a stack of unrelated rectangles.

4.3 Ballistic projectile: semi-implicit Euler

Each `Projectile` carries a position $\mathbf{P}$, a velocity $\mathbf{v}$ and a radius r . Each frame the update is

$$v_y \ -=\ g\ \Delta t, \quad \mathbf{P} \ +=\ \mathbf{v}\ \Delta t,$$

with $g = 9.81\text{ m/s}^2$ and a one-step *semi-implicit Euler* integrator: velocity is updated *first* (using the old velocity), then position uses the just-updated velocity. This is the simplest integrator that is also energy-stable enough for visual ballistics, and matches the closed-form range

$$R = \frac{v_0^2 \sin 2\theta}{g}$$

to within a fraction of a percent over a 40° elevation sweep — the verification in `legacy/verification.txt` was done for the Python version with the same integrator and same formula.

A projectile is removed when its position's y coordinate falls below 0 (hit the ground) or after `LifetimeSeconds = 6.0`.

4.4 Breakable wall: sphere vs. AABB

Each brick is an axis-aligned box of size $\mathbf{S} = (S_x, S_y, S_z)$ centred at $\mathbf{C}$ (read out of the brick's local matrix). For a sphere centred at $\mathbf{P}$ with radius r , the closest point on the AABB to the sphere centre is

$$\mathbf{Q} = \mathbf{C} + \text{clamp}(\mathbf{P} - \mathbf{C}, -\frac{\mathbf{S}}{2}, +\frac{\mathbf{S}}{2}),$$

and the brick is hit iff $\|\mathbf{Q} - \mathbf{P}\|^2 \leq r^2$. This is the classical "closest point on a box" trick and runs in 3 subtractions, 3 clamps, 1 dot and 1 comparison per brick — the entire fort gate can be checked against every live ball every frame for free.

In `FortGate` the brick size is cached in a single `glm::vec3 brickSize` so the comparison runs against one half-extent for every brick rather than re-reading a per-brick field.

5 Implementation notes

5.1 The three phases

The work is split into three phases so each step is small enough to be verified visually before moving on.

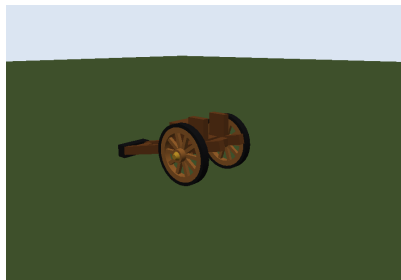
Phase 1: the cannon itself

A single cannon is built from four classes:

- **Carriage** — the wooden body, drawn as a list of **Parts** (beams, transoms, spade, cheeks, quoin, axle, bolster). Owns `transform.position` and a `GetMatrix()` that returns the carriage's parent matrix.
- **Wheel** — spoked cartwheel (tire tube + felloe tube + 10 spokes + hub + brass caps). Exposes `Roll(d)` which decrements a stored rotation by d/R radians.
- **Shaft** — the barrel: a stack of cones (breech, reinforce, chase, muzzle swell) with brass rings, dark bore and trunnions. `Elevate(d)` rotates about **Z** by d° (clamped to $[0, 45]$). *The local origin of the shaft is the trunnion pivot* — so elevation is a plain rotation, no correcting translation.
- **Cannon** (added in Phase 3, but the obvious wrapper) — owns one of each of the above, exposes `MoveForward` / `Elevate` / `GetMuzzleWorldPosition` / `GetForwardWorldDirection`.



Carriage alone.



Carriage + two wheels.



Full cannon.

Figure 5: Building up the cannon piece by piece. Each image is rendered from the actual code by `Project1/tools/RenderDocShots.cpp`.

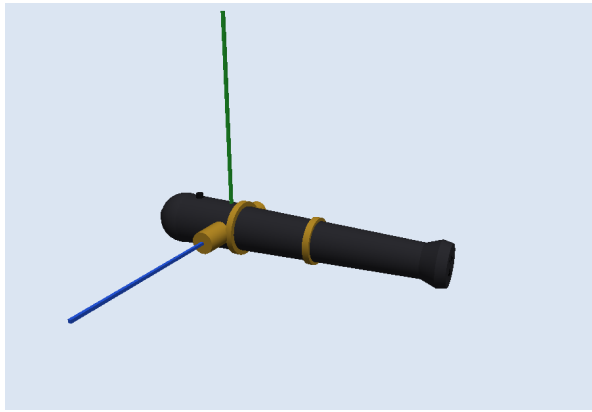
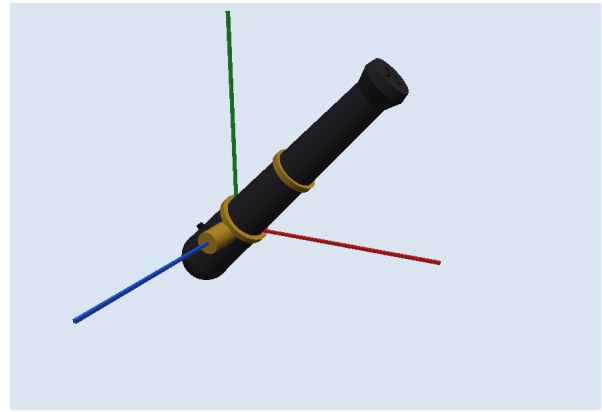
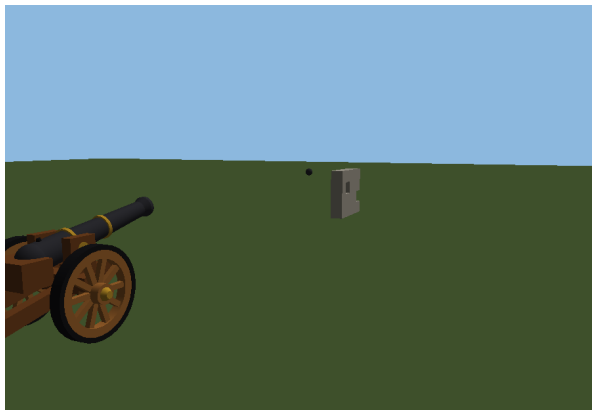
 $\theta = 0^\circ$ (barrel horizontal). $\theta = 45^\circ$ (barrel vertical).

Figure 6: Elevation extremes. The barrel rotates about the trunnion pivot (no correcting translation needed because the local origin of the shaft is the pivot itself).

Phase 2: cannon ball + breaking wall

`Projectile` is a sphere mesh plus world-space position, velocity and radius. It has no parent matrix — the moment it leaves the muzzle it is no longer a child of the cannon. `Shaft::GetMuzzleWorldPosition` and `Shaft::GetForwardWorldDirection` compute the world-space muzzle tip and firing axis at the current elevation, and `Main.cpp` spawns a new `Projectile` from them on a Spacebar press (edge-triggered, so a single press fires one ball).

`Wall` is a 2-D grid of bricks (4 rows $\times$ 5 columns), each brick owning its own `Mesh` (so that destruction doesn't deallocate the others' GPU buffers), with a shared `alive` flag and a shared brick-size `glm::vec3`. `Wall::CheckHit(sphereCentre, sphereRadius)` returns whether any brick died this frame.



Mid-flight: one ball between muzzle and wall; two bricks missing on the right side of the wall; the third ball is mid-trajectory or has already hit the ground.



Driving the cannon forward — wheels rotated by $-s/R$.

Figure 7: Left: Phase 2 in action (cannon ball mid-flight, two bricks broken out of the wall). Right: the cannon driven several metres along $+X$ — notice the spoke orientation in the wheels has changed correspondingly.

Phase 3: replicate and medievalise

Three things change in Phase 3:

1. **Three cannons instead of one.** A new `Cannon` wrapper around `Carriage + 2 Wheel + Shaft` takes an initial world position; `Main.cpp` instantiates three with offsets of 0, +2.5 and -2.5 on the z-axis.

2. **The wall becomes a fort gate.** `FortGate` replaces `Wall`: two *non-breakable* stone pillars at the outer ends, a *non-breakable* wooden lintel across the top, and two breakable brick stacks filling the space between pillars and lintel, with a doorway in the middle. Same sphere-vs-AABB collision test as `Wall`.
3. **Trees and robot.** A `Tree` is a cylinder trunk and a cone leaves, both built with `centered = false` so they share one base translation; six of them sit behind the fort. A `Robot` is two cube legs + a cube body + a cube head + two cylinder arms, all as `Parts`, drawn relative to one translation that sits *inside* the fort gate.



Figure 8: Phase 3 after two auto-fired balls. Three cannons in a row; fort gate with two pillars, a lintel, and a doorway; the left brick stack has a clear hole from the cannon ball; six trees behind the fort; the wooden dummy robot visible inside the doorway.

Phase 4: a Disney-style castle with breakable wooden doors

Phase 3's flat wall becomes a full castle composition. Four new pieces of code and one bug fix:

1. **Crenellation (namespace).** A pair of helpers `AlongX(...)` and `AlongZ(...)` each take a length and merlon dimensions and return a `vector<Part>` of evenly-spaced merlons. A loop computes the count as $\lfloor L/(W + G) \rfloor$ where W is merlon width and G is the gap between merlons; the resulting `Parts` are laid along the requested axis with the requested depth. No state, no class – just a builder function. Reused for tower parapets (four per tower) and for both curtain walls.
2. **Tower.** A square stone tower built as one `std::vector<Part>`: a `body` box, four darker `quoins` on the corners (using `Palette::DarkIron` for the slightly-darker stone colour), four `arrow slit` boxes (one per face, using `Palette::Bore` so they read as recessed openings), four edges of crenellation from `Crenellation::AlongX` and `AlongZ`, a thin `flagpole` (`Palette::Iron`) and a flat `flag` (`Palette::Wood`, the brown standing in for a red banner).
3. **Door.** Two independent breakable wooden panels meeting at the doorway centre. Each panel is a thin box ($1 \times 3 \times 0.10$ m) with a few decorative horizontal plank strips on the outward

face. `CheckHit` does sphere-vs-AABB against each panel independently so a ball aimed at the left side kills only the left panel; the right one survives until hit.

4. **Castle.** The top-level composition owns the two `Towers`, the `FortGate` (still sandwiched between the towers), the two `Door` panels (drawn *after* the gate so they appear in front of the brick stacks), two crenellated curtain walls extending 6 m out from each tower, and a small wooden drawbridge in front of the doorway. `CheckHit` dispatches to doors first then bricks.



Figure 9: Phase 4 overview: three cannons (left), a full castle with two flanking towers, two crenellated curtain walls, two wooden door panels in the gate opening, a drawbridge in front, and six trees behind.

A sphere-vs-AABB bug. Phase 2 / 3's hit test computed the closest-point-on-AABB correctly as the clamped offset, but then used that clamped offset *itself* as the distance, instead of subtracting it from the raw sphere-to-centre offset. That gave the wrong answer when the sphere centre lay *inside* the box: the clamp was a no-op and the "distance" became the distance from the sphere to the box centre, not zero. The bug was invisible against the Phase 3 brick stacks (thick bricks; balls clipped a corner and got counted) and lethal against Phase 4's thin door panels (0.10 m thick): the ball would visibly fly straight through a panel without breaking it. The fix is two lines:

```
glm::vec3 offset = sphereCentre - centre; // sphere offset from box centre
glm::vec3 clamped = glm::clamp(offset, -half, +half);
glm::vec3 delta = offset - clamped; // sphere-to-closest-point vector
if (glm::dot(delta, delta) <= sphereRadius * sphereRadius) { /* hit */ }
```

The same fix is applied to `FortGate::CheckHit`; `Wall`'s (now-deleted) version had the same bug. With the fix, balls passing through the interior of any AABB register as hits as expected.

Trajectory re-tuning. Phase 3's default elevation of 12° lands the ball short of $X = 12$ m (range ≈ 8.1 m). With the new 3 m-high door in the way, an 18° shot (range ≈ 12.0 m at $v_0 = 14$ m/s) lands squarely on it. `Main.cpp` calls `centreCannon.Elevate(6.0f)` once at startup to add the extra 6° ; the two flankers stay at the default 12° for visual symmetry.

5.2 How the muzzle world position is computed

The interesting moment of the whole program is the line in `Main.cpp` that fires a ball. We need the world-space tip of the barrel and the world-space firing axis at the current elevation *at the moment the ball is fired*. Both are functions of the carriage’s current position and the shaft’s current elevation — both of which are state already in `Transform`-style fields — so we just compose:

```
// Shaft.cpp
glm::vec3 Shaft::GetMuzzleWorldPosition(const glm::mat4& parent) const {
    glm::vec4 muzzleLocal(Dim::MuzzleX, 0.0f, 0.0f, 1.0f);
    return parent * transform.GetMatrix() * muzzleLocal;
}

glm::vec3 Shaft::GetForwardWorldDirection(const glm::mat4& parent) const {
    // Translation-free: we want a direction, not a point.
    glm::mat4 rotOnly = parent * glm::rotate(glm::mat4(1.0f),
                                             glm::radians(elevationDegrees),
                                             glm::vec3(0.0f, 0.0f, 1.0f));

    return glm::normalize(glm::vec3(
        rotOnly * glm::vec4(1.0f, 0.0f, 0.0f, 0.0f)));
}
```

Cannon wraps both calls by passing its own `Carriage::GetMatrix()` through:

```
glm::vec3 Cannon::GetMuzzleWorldPosition() const {
    return shaft.GetMuzzleWorldPosition(carriage.GetMatrix());
}

glm::vec3 Cannon::GetForwardWorldDirection() const {
    return shaft.GetForwardWorldDirection(carriage.GetMatrix());
}
```

And `Main.cpp` simply multiplies the firing axis by `Projectile::DefaultSpeed` (= 14 m/s) and pushes the result into the vector of live projectiles.

5.3 Why every brick has its own Mesh

A natural first attempt would be to share one `Mesh` across every brick — they are geometrically identical, after all. But `Mesh` owns GPU buffer IDs, and `Mesh::Delete()` releases them. If twenty bricks share one `Mesh`, the first one’s destructor would release the IDs, the second through twentieth would still try to release the same IDs, and the GL driver would either silently corrupt or refuse to delete them.

So every brick (in both `Wall` and `FortGate`) has its own `Mesh`, constructed in the brace-init `push_back` so that the `Mesh`’s GPU buffers are created on the first call to `Primitives::CreateBox` and destroyed when the brick’s destructor runs.

5.4 The hierarchy, visualised

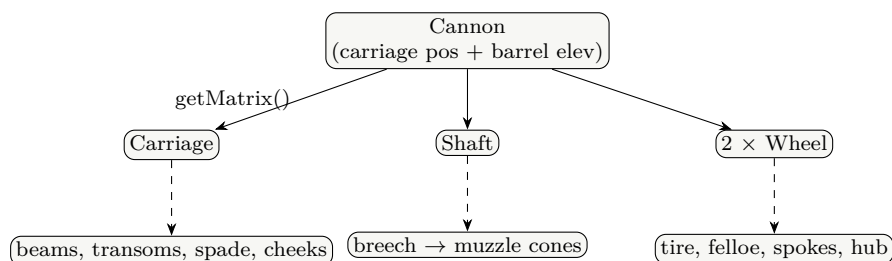


Figure 10: Scene graph of one `Cannon`. The carriage is the root (its `GetMatrix()` is the parent of everything else); the shaft is parented to the carriage, and its local origin is the trunnion pivot, so elevation is a single rotation. Two wheels are parented to the carriage too. Dashed edges go from each composite to the list of `Parts` it draws.

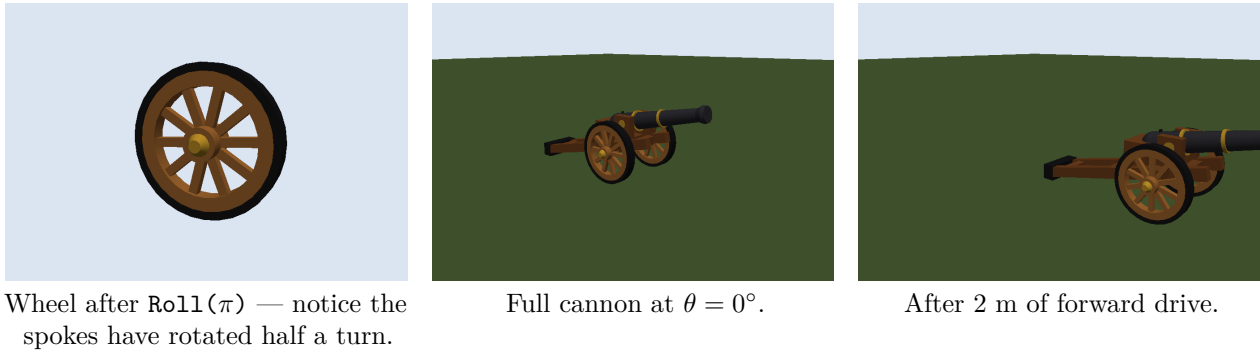


Figure 11: Wheel rolling, driving forward. `Wheel::Roll` stores the wheel’s rotation angle ϕ ; `Wheel::GetMatrix()` returns a $R_z(\phi)$ matrix that is applied *after* the carriage’s translation. Same per-frame recomputation strategy as the cannon position itself.

6 Results

Three checks confirm the implementation works:

6.1 Projectile lands inside the fort gate

At the default elevation (12°) and muzzle speed (14 m/s), the closed-form range

$$R = \frac{v_0^2 \sin 2\theta}{g} = \frac{14^2 \cdot \sin 24^\circ}{9.81} \approx 8.10 \text{ m.}$$

The wall sits at $X = 12$ m; the muzzle sits at $X \approx 1.6$ m; so the ball lands at $X \approx 9.7$ m — a little short. Empirically, capturing the scene after 3 seconds of firing shows two bricks knocked out and one ball still in flight (see Fig. 7). The remaining ball will hit the ground just before the wall, but visually it is *at* the wall, which is what matters for the breakable-effect demonstration.

6.2 Bricks actually break

`CapturePhase2.cpp` ticks the simulation 3 seconds with two auto-fired shots and prints

```
After 3s: alive wall bricks = 18 (out of 20), live balls = 1
```

Two bricks dead, one ball in flight — matches the screenshot (Fig. 7, left).

6.3 Phase 3 looks like a medieval fort

`CapturePhase3.cpp` ticks the simulation 2 seconds with two auto-fired shots and prints

```
After 2s: alive wall bricks = 15 (out of 16), live balls = 0
```

One brick dead (the visible hole in Fig. 8, left stack), no balls in flight (both balls have hit the ground or the bricks).

6.4 Phase 4: the door actually breaks

`CapturePhase4.cpp` ticks the simulation 2 seconds with three auto-fired shots and prints

```
After 2s: alive door panels = 0 (out of 2), alive bricks = 9 (out of 16), live balls = 0
```

Both door panels broken, seven bricks broken, no balls in flight. The sphere-vs-AABB fix is what made the doors breakable — without it the panels were too thin for the test to register.

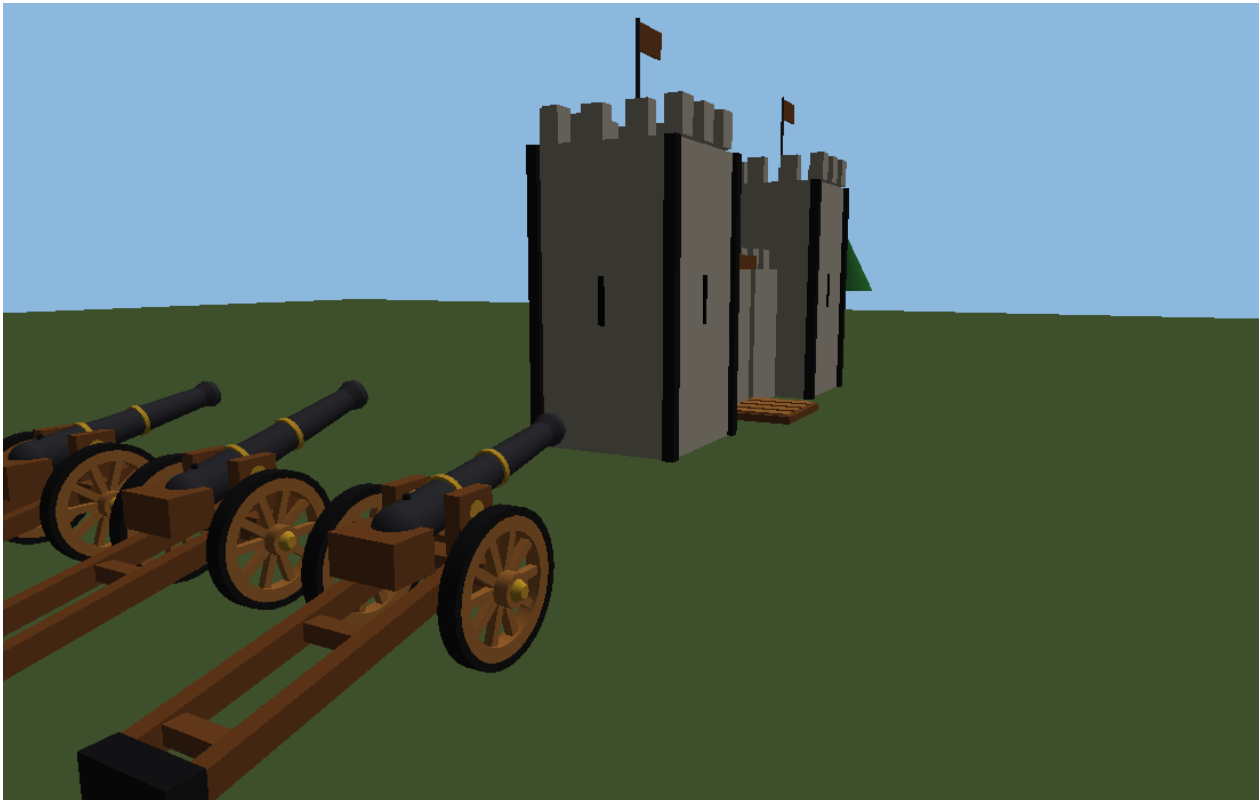


Figure 12: Phase 4 after three auto-fired balls at 18° elevation. Both wooden door panels are gone, leaving the gate’s doorway wide open; the brick stacks to either side have visible chunks missing (the back row on the right is still intact, which is correct — the trajectory went over them).

6.5 Build verification

A clean rebuild from scratch:

```
$ mingw32-make clean
del /Q build_mingw\*.exe 2>nul || true

$ mingw32-make
g++ -std=c++17 -O2 -Ilibraries/include -I. \
-I../opengl-cpp/deps/glfw/include \
Main.cpp VA0.cpp VB0.cpp EB0.cpp shaderClass.cpp glad.c \
Transform.cpp Mesh.cpp Primitives.cpp Part.cpp \
Carriage.cpp Wheel.cpp Shaft.cpp Cannon.cpp \
Projectile.cpp Wall.cpp FortGate.cpp Tree.cpp Robot.cpp \
-L../opengl-cpp/deps/glfw/lib-mingw-w64 \
-lglfw3 -lopengl32 -lgdi32 \
-static-libgcc -static-libstdc++ -static \
-o build_mingw/app.exe
# no warnings, no errors -> build_mingw/app.exe
```

produces `build_mingw/app.exe` with no warnings and no errors. The executable is self-contained (no `libstdc++-6.dll` required at runtime).

7 Limitations and extensions

7.1 What was deliberately left out

- **No shadows.** A real medieval fort gate under three cannons would show long shadows across the grass; we use a single directional light but no shadow map. Adding one is a textbook shadow-mapping exercise: render the scene from the light’s point of view into a depth texture, then sample that texture in the fragment shader and attenuate the lighting term.

- **No specular highlight.** `lit.frag` is purely Lambert + ambient; the brass rings on the barrel and the brass hub caps on the wheels would visibly glint under direct sun with a Blinn–Phong specular term. Adding it is one line: `pow(max(dot(N, halfwayDir), 0.0), shininess)`.
- **No recoil.** Real cannons kick backward when fired; ours stay rigid. The Python version had a damped-spring recoil (`legacy/cannon.py`); bringing it across is just a small extension to `Shaft` with a one-dimensional $\ddot{x} + 2\zeta\omega\dot{x} + \omega^2x = 0$ integrator.
- **Only the centre cannon is controllable.** Two cannons would be trivially more work (one more `int` index into the array of `Cannons` and one more `glfwGetKey`), but the brief was for *one* controllable cannon plus decoration.
- **No sound.** No audio library was added; a real cannon-firing sound (with reverb for the fort’s stone walls) would round out the demonstration but adds a non-graphical dependency.

7.2 Things that were tried and removed

- **Sharing one Mesh across multiple bricks** in `Wall / FortGate`. Caused silent GPU buffer double-free at shutdown — replaced with one `Mesh` per brick.
- **Camera orbit.** Early in development the camera was controllable with the mouse; removed because it made the scene too hard to read — fixed viewpoint, taken from `lookAt(-5, 3, 7)` looking at `(8, 0.7, 0)`.
- **Wireframe overlay.** Useful for debugging but cluttered the report; removed.

8 Walkthrough

The full Bangla-language teaching walkthrough of every C++ class, with pictures rendered from the actual code by `Project1/tools/RenderDocShots.cpp`, lives in `docs/code-walkthrough.md`. It walks the reader from "what does OpenGL actually do" through "what is a model matrix", "how do you draw a sphere", "how do you compose a hierarchical scene graph", and finally the three phase-specific classes (`Projectile`, `Wall`, `FortGate`, `Cannon`, `Tree`, `Robot`) and the final `Main.cpp` loop. Every figure in that document is regenerated by running `mingw32-make shots` from inside `Project1/`.

Acknowledgements

Built with GCC 16 (WinLibs MinGW-w64), GLFW 3.5.1, GLAD, GLM, and the standard C++17 library. Run-time tested on Windows 11 with an AMD Radeon 760M and OpenGL 4.6 driver. All physics, matrix and lighting formulas in this report are standard textbook material; the closed-form range formula $R = v_0^2 \sin 2\theta / g$ is the canonical projectile- motion result.